

CONTROL AND OBSERVABILITY

Configure permissions

Control how your agent uses tools with permission modes, hooks, and declarative allow/deny rules.

The Claude Agent SDK provides permission controls to manage how Claude uses tools. Use permission modes and rules to define what's allowed automatically, and the `canUseTool` [callback](#) to handle everything else at runtime.

! This page covers permission modes and rules. To build interactive approval flows where users approve or deny tool requests at runtime, see [Handle approvals and user input](#).

How permissions are evaluated

When Claude requests a tool, the SDK checks permissions in this order:

1 Hooks

Run [hooks](#) first. A hook can deny the call outright or pass it on. A hook that returns `allow` does not skip the deny and ask rules below; those are evaluated regardless of the hook result.

2 Deny rules

Check `deny` rules (from `disallowed_tools` and [settings.json](#)). If a deny rule matches, the tool is blocked, even in `bypassPermissions` mode. Bare-name deny rules like `Bash` remove the tool from Claude's context before this evaluation begins, so only scoped rules like `Bash(rm *)` are checked at this step.

3 Ask rules

Check `ask` rules from [settings.json](#). If an ask rule matches, the call falls through to your `canUseTool` [callback](#) for confirmation, even in `bypassPermissions` mode. In `dontAsk` mode a matching ask rule is denied instead, because that mode never prompts.

4 Permission mode

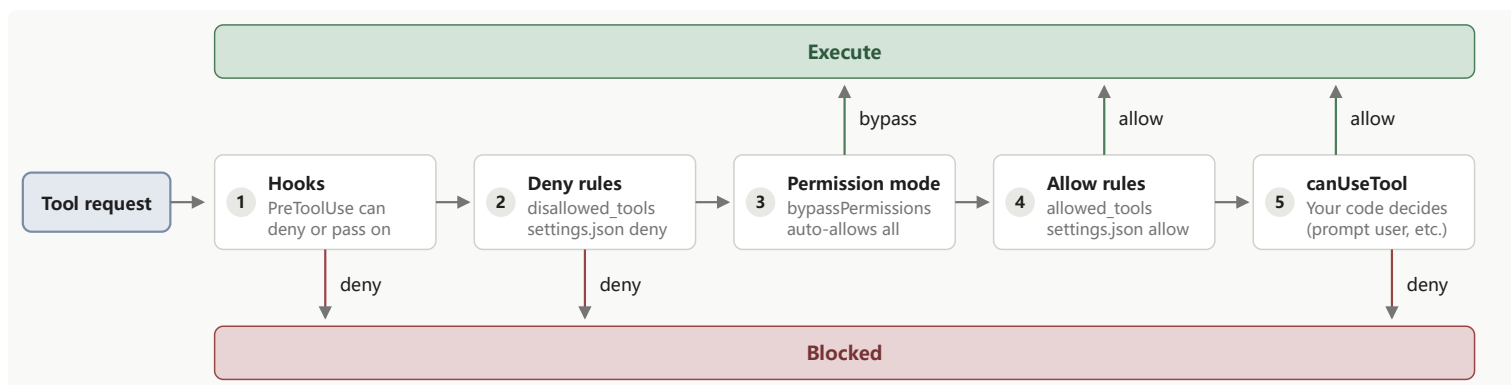
Apply the active **permission mode**. `bypassPermissions` approves everything that reaches this step. `acceptEdits` approves file operations. `plan` routes file-edit and shell-write tools to your `canUseTool` callback regardless of allow rules, so write operations cannot be auto-approved while planning. Other modes fall through.

5 Allow rules

Check `allow` rules (from `allowed_tools` and `settings.json`). If a rule matches, the tool is approved.

6 canUseTool callback

If not resolved by any of the above, call your **canUseTool callback** for a decision. In `dontAsk` mode, this step is skipped and the tool is denied.



This page focuses on **allow and deny rules** and **permission modes**. For the other steps:

- **Hooks:** run custom code to allow, deny, or modify tool requests. See [Control execution with hooks](#).
- **canUseTool callback:** prompt users for approval at runtime. See [Handle approvals and user input](#).

Allow and deny rules

`allowed_tools` and `disallowed_tools` (TypeScript: `allowedTools` / `disallowedTools`) add entries to the allow and deny rule lists in the evaluation flow above. Allow rules only affect approval: a tool not listed in `allowed_tools` is still available to Claude and falls through to the permission mode. Deny rules behave differently depending on whether they name a tool or scope a pattern within one.

Option	Effect
<code>allowed_tools=["Read", "Grep"]</code>	<code>Read</code> and <code>Grep</code> are auto-approved. Tools not listed here still exist and fall through to the permission mode and <code>canUseTool</code> .
<code>disallowed_tools=["Bash"]</code>	The <code>Bash</code> tool definition is removed from the request. Claude does not see the tool and cannot attempt it.
<code>disallowed_tools=["Bash(rm *)"]</code>	<code>Bash</code> stays available. Calls matching <code>rm *</code> are denied in every permission mode, including <code>bypassPermissions</code> . Other <code>Bash</code> calls fall through to the permission mode.
<code>disallowed_tools=["*"]</code>	Every tool definition is removed from the request. Tool-name globs are supported in deny rules: <code>"*"</code> matches every tool and <code>"mcp__*"</code> matches every MCP tool across all servers.

Allow rules accept tool-name globs only after a literal `mcp__<server>__` prefix. The server segment must be glob-free so the rule names a specific server you configured: `mcp__puppeteer__*` matches every tool from the `puppeteer` server, and `mcp__github__get_*` matches its `get_` tools. An unanchored entry like `allowed_tools=["*"]` or `allowed_tools=["mcp__*"]` is ignored with a startup warning and does not auto-approve anything.

For a locked-down agent, pair `allowedTools` with `permissionMode: "dontAsk"` . Listed tools are approved; anything else is denied outright instead of prompting:

```
const options = {
  allowedTools: ["Read", "Glob", "Grep"],
  permissionMode: "dontAsk"
};
```

⚠️ `allowed_tools` **does not constrain** `bypassPermissions` . `allowed_tools` only pre-approves the tools you list. Unlisted tools are not matched by any allow rule and fall through to the permission mode, where `bypassPermissions` approves them. Setting `allowed_tools=["Read"]` alongside `permission_mode="bypassPermissions"` still approves every tool, including `Bash` , `Write` , and `Edit` . If you need `bypassPermissions` but want specific tools blocked, use `disallowed_tools` .

You can also configure allow, deny, and ask rules declaratively in `.claude/settings.json` . These rules are read when the `project` setting source is enabled, which it is for default `query()` options. If you set `setting_sources` (TypeScript: `settingSources`) explicitly, include `"project"` for them to apply. See [Permission settings](#) for the rule syntax.

Permission modes

Permission modes provide global control over how Claude uses tools. You can set the permission mode when calling `query()` or change it dynamically during streaming sessions.

Available modes

The SDK supports these permission modes:

Mode	Description	Tool behavior
<code>default</code>	Standard permission behavior	No auto-approvals; unmatched tools trigger your <code>canUseTool</code> callback
<code>dontAsk</code>	Deny instead of prompting	Anything not pre-approved by <code>allowed_tools</code> or rules is denied; <code>canUseTool</code> is never called
<code>acceptEdits</code>	Auto-accept file edits	File edits and <u>filesystem operations</u> (<code>mkdir</code> , <code>rm</code> , <code>mv</code> , etc.) are automatically approved
<code>bypassPermissions</code>	Bypass permission checks	Tools run without permission prompts, unless an explicit <u>ask rule</u> matches (use with caution)
<code>plan</code>	Planning mode	Claude explores and plans without editing your source files; file edits are never auto-approved and prompt through your <code>canUseTool</code> callback
<code>auto</code> (TypeScript only)	Model-classified approvals	A model classifier approves or denies each tool call. See <u>Auto mode</u> for availability

 **Subagent inheritance:** When the parent uses `bypassPermissions`, `acceptEdits`, or `auto`, all subagents inherit that mode and it cannot be overridden per subagent. Subagents may have different system prompts and less constrained behavior than your main agent, so inheriting `bypassPermissions` grants them full, autonomous system access. An explicit ask rule still forces a prompt.

Set permission mode

You can set the permission mode once when starting a query, or change it dynamically while the session is active.

At query time

Pass `permission_mode` (Python) or `permissionMode` (TypeScript) when creating a query. This mode applies for the entire session unless changed dynamically.

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    async for message in query(
        prompt="Help me refactor this code",
        options=ClaudeAgentOptions(
            permission_mode="default", # Set the mode here
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)

asyncio.run(main())
```

During streaming

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    async for message in query(
        prompt="Help me refactor this code",
        options=ClaudeAgentOptions(
            permission_mode="default", # Set the mode here
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)

asyncio.run(main())
```

TypeScript

Call `set_permission_mode()` (Python) or `setPermissionMode()` (TypeScript) to change the mode mid-session. The new mode takes effect immediately for all subsequent tool requests. This lets you start restrictive and loosen permissions as trust builds, for example switching to `acceptEdits` after reviewing Claude's initial approach.

Python TypeScript

```
import asyncio
from claude_agent_sdk import ClaudeSDKClient, ClaudeAgentOptions

async def main():
    async with ClaudeSDKClient(
        options=ClaudeAgentOptions(
            permission_mode="default", # Start in default mode
        )
    ) as client:
        await client.query("Help me refactor this code")

        # Change mode dynamically mid-session
        await client.set_permission_mode("acceptEdits")

        # Process messages with the new permission mode
        async for message in client.receive_response():
            if hasattr(message, "result"):
                print(message.result)

asyncio.run(main())
```

```

import asyncio
from claude_agent_sdk import ClaudeSDKClient, ClaudeAgentOptions

async def main():
    async with ClaudeSDKClient(
        options=ClaudeAgentOptions(
            permission_mode="default", # Start in default mode
        )
    ) as client:
        await client.query("Help me refactor this code")

        # Change mode dynamically mid-session
        await client.set_permission_mode("acceptEdits")

        # Process messages with the new permission mode
        async for message in client.receive_response():
            if hasattr(message, "result"):
                print(message.result)

asyncio.run(main())

```

TypeScript

Mode details

Accept edits mode (`acceptEdits`)

Auto-approves file operations so Claude can edit code without prompting. Other tools (like Bash commands that aren't filesystem operations) still require normal permissions.

Auto-approved operations:

- File edits (Edit, Write tools)
- Filesystem commands: `mkdir` , `touch` , `rm` , `rmdir` , `mv` , `cp` , `sed`

Both apply only to paths inside the working directory or `additionalDirectories` . Paths outside that scope and writes to protected paths still prompt.

Use when: you trust Claude's edits and want faster iteration, such as during prototyping or when working in an isolated directory.

Don't ask mode (`dontAsk`)

Converts any permission prompt into a denial. Tools pre-approved by `allowed_tools` , `settings.json` allow rules, or a hook run as normal. Everything else is denied without calling `canUseTool` .

Use when: you want a fixed, explicit tool surface for a headless agent and prefer a hard deny over silent reliance on `canUseTool` being absent.

Bypass permissions mode (`bypassPermissions`)

Auto-approves all tool uses without prompts. Hooks still execute and can block operations if needed.

 Use with extreme caution. Claude has full system access in this mode. Only use in controlled environments where you trust all possible operations.

`allowed_tools` does not constrain this mode. Every tool is approved, not just the ones you listed. Deny rules (`disallowed_tools`), explicit `ask` rules, and hooks are evaluated before the mode check and can still block a tool.

Plan mode (`plan`)

Claude explores the codebase and produces a plan without editing your source files. Read-only tools run as in default mode. File edits are never auto-approved in plan mode, even when an allow rule matches. They prompt through your `canUseTool` callback instead. Claude may use

`AskUserQuestion` to clarify requirements before finalizing the plan. See [Handle approvals and user input](#) for handling these prompts.

Use when: you want Claude to propose changes without executing them, such as during code review or when you need to approve changes before they're made.

Related resources

For the other steps in the permission evaluation flow:

- [Handle approvals and user input](#): interactive approval prompts and clarifying questions
- [Hooks guide](#): run custom code at key points in the agent lifecycle

- Permission rules: declarative allow/deny rules in `settings.json`